

PROTOCOLO UDP CONFIABLE PARA RED NEURONAL DISTRIBUIDA

Proyecto de Redes y Comunicaciones

18 de mayo de 2026

Índice

1. Descripción General	3
2. Configuración de la Red Neuronal	3
3. Arquitectura General	3
4. Responsabilidades del Sistema	4
4.1. Python	4
4.2. Servidor Principal	4
4.3. Workers	4
5. Transferencia Confiable de Datos (RDT)	5
6. Características del Protocolo RDT	5
7. Parámetros del Protocolo	5
8. Formato del Paquete UDP	6
8.1. Estructura del paquete	6
8.2. Descripción de campos	6
9. Tipos de Paquete	7
10. Tipos de Operación	7
11. Campos Usados por Cada Tipo de Paquete	7
12. Integridad de Paquetes	7
12.1. Si el CRC32 coincide	8
12.2. Si el CRC32 falla	8
13. Números de Secuencia, ACK y Ventana	8
14. Timeouts y Reintentos	9

15.Estados del Emisor y Receptor	9
15.1. Estados del Emisor	9
15.2. Estados del Receptor	9
16.Secuencia de Comunicación	10
17.Distribución de Matrices	11
18.Serialización de Matrices	11
19.Flujo General del Sistema	11
20.Operaciones Matriciales	12
20.1. Distribución de Multiplicación Matricial	12
21.Integración Python y C++	12
22.Posibles Mejoras Futuras	12
23.Tecnologías Utilizadas	12
24.Conclusión	13

1. Descripción General

El objetivo del proyecto es implementar un sistema de Red Neuronal Distribuida (DNN) donde:

- La red neuronal se define y entrena en Python.
- Las operaciones matriciales se distribuyen utilizando C++.
- La comunicación entre nodos utiliza UDP.
- La confiabilidad se implementa manualmente mediante un protocolo RDT (Reliable Data Transfer).

La arquitectura del sistema está compuesta por:

- Un servidor principal (Main Server / Boss)
- Múltiples servidores esclavos (Workers / Slaves)

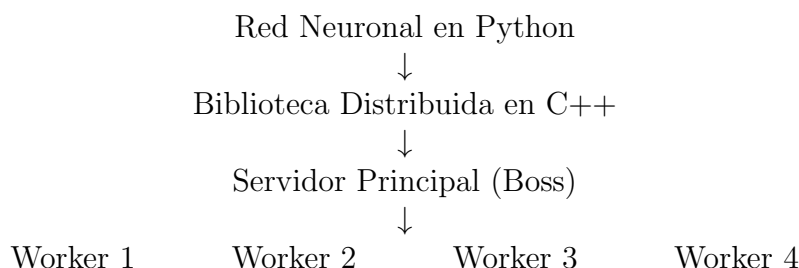
El servidor principal distribuye operaciones matriciales a los workers, recibe resultados y reconstruye la matriz final.

2. Configuración de la Red Neuronal

La configuración de la Red Neural es la siguiente:

Parámetro	Valor
Capas mínimas	3
Neuronas ocultas	200
Tamaño de matrices	200x200
Batch size	1
Protocolo de comunicación	UDP
Capa de confiabilidad	Go-Back-N RDT
Tipo de dato	float32
Formato binario de matrices	little-endian
Endianness del protocolo	big-endian / network byte order
Tamaño de ventana inicial	8 paquetes
Timeout inicial	500 ms
Máximo de reintentos	5

3. Arquitectura General



4. Responsabilidades del Sistema

4.1. Python

Python será responsable de:

- Cargar el dataset
- Definir la red neuronal
- Ejecutar el entrenamiento
- Llamar a la biblioteca distribuida en C++
- Recibir matrices procesadas

La integración entre Python y C++ se realizará mediante pybind11.

4.2. Servidor Principal

El servidor principal será responsable de:

- Recibir matrices desde Python
- Dividir matrices en bloques
- Serializar matrices
- Construir paquetes UDP
- Administrar ventana Go-Back-N
- Enviar tareas a los workers
- Administrar ACK y retransmisiones
- Reconstruir resultados

4.3. Workers

Los workers serán responsables de:

- Recibir paquetes UDP
- Verificar integridad
- Reconstruir fragmentos en orden
- Detectar pérdidas y duplicados
- Descartar paquetes fuera de orden
- Realizar operaciones matriciales
- Retornar resultados

5. Transferencia Confiable de Datos (RDT)

UDP no garantiza:

- Entrega de paquetes
- Orden correcto
- Integridad
- Protección contra duplicados

Por esta razón se implementará un protocolo RDT sobre UDP utilizando Go-Back-N.

6. Características del Protocolo RDT

El protocolo implementará el mecanismo Go-Back-N sobre UDP. Go-Back-N permitirá enviar múltiples paquetes consecutivos sin esperar una confirmación inmediata por cada uno de ellos.

Característica	Descripción
ACK acumulativo	Confirma el último paquete recibido correctamente en orden
Ventana deslizante	Permite mantener varios paquetes en tránsito simultáneamente
Número de secuencia	Detecta pérdidas, duplicados y mantiene el orden
Timeout	Detecta pérdida de paquetes o ACKs
Retransmisión Go-Back-N	Retransmite todos los paquetes desde el paquete posterior al último ACK confirmado
CRC32	Detecta corrupción tanto en el encabezado como en el payload
Recepción en orden	El receptor solo acepta el paquete esperado y descarta paquetes fuera de orden

7. Parámetros del Protocolo

Los siguientes valores serán utilizados como configuración inicial del protocolo:

Parámetro	Valor
Endianness del header	Big-endian / network byte order
Tamaño máximo del datagrama UDP	512 bytes
Tamaño máximo del payload	484 bytes
Tamaño inicial de ventana	8 paquetes
Timeout inicial	500 ms
Máximo de reintentos	5
Primer número de secuencia DATA	0
CRC	CRC32 sobre header y payload válido

Todos los campos numéricos del encabezado del protocolo serán serializados en big-endian, también conocido como network byte order. Las matrices, por otro lado, mantendrán su representación interna como arreglos de tipo float32 en formato little-endian.

Si un paquete o conjunto de paquetes supera el máximo de reintentos definido, la transferencia se considerará fallida y será cancelada.

8. Formato del Paquete UDP

Cada datagrama UDP tendrá un tamaño máximo de 500 bytes.

8.1. Estructura del paquete

Campo	Tamaño
Tipo de paquete	1 byte
Número de secuencia	4 bytes
Número ACK	4 bytes
ID de matriz	4 bytes
Número de fragmento	4 bytes
Total de fragmentos	4 bytes
Tipo de operación	1 byte
Tamaño útil del payload	2 bytes
CRC32	4 bytes
Payload	472 bytes

8.2. Descripción de campos

- **Tipo de paquete:** Identifica si el paquete corresponde a DATA, ACK, NACK, START o END.
- **Número de secuencia:** Identifica el orden lógico del paquete dentro de la transferencia. Permite detectar pérdidas, duplicados y mantener el orden correcto.
- **Número ACK:** Indica el último número de secuencia recibido correctamente y en orden. Este campo se utiliza principalmente en paquetes ACK. Por ejemplo, si el receptor recibió correctamente los paquetes 0, 1 y 2, enviará un ACK con valor 2.
- **ID de matriz:** Identifica la matriz asociada al paquete.
- **Número de fragmento:** Indica la posición del fragmento dentro de la matriz.
- **Total de fragmentos:** Permite reconstruir completamente la matriz original.
- **Tipo de operación:** Define la operación matricial solicitada.
- **Tamaño útil del payload:** Indica cuántos bytes válidos contiene el payload. Este valor no puede superar 484 bytes.

- **CRC32:** Código utilizado para detectar corrupción en el encabezado y en el payload. Para calcularlo, el campo CRC32 se coloca temporalmente en cero y luego se calcula el CRC32 sobre todo el encabezado más el payload válido.
- **Payload:** Contiene fragmentos serializados de matrices.

9. Tipos de Paquete

Tipo	Valor numérico	Significado
DATA	0x01	Paquete de datos
ACK	0x02	ACK acumulativo
NACK	0x03	Solicitud explícita de retransmisión
START	0x04	Inicio de transmisión
END	0x05	Fin de transmisión

10. Tipos de Operación

Operación	Valor numérico	Significado
ADD	0x01	Suma matricial
MUL	0x02	Multiplicación matricial
AVG	0x03	Promedio matricial
NONE	0x00	Sin operación asociada

11. Campos Usados por Cada Tipo de Paquete

No todos los tipos de paquete utilizan todos los campos del encabezado. Los campos no utilizados deberán enviarse con valor cero.

Tipo de paquete	Campos principales utilizados
START	Tipo de paquete, ID de matriz, total de fragmentos, tipo de operación, CRC32
DATA	Tipo de paquete, número de secuencia, ID de matriz, número de fragmento, total de fragmentos, tipo de operación, tamaño útil del payload, CRC32, payload
ACK	Tipo de paquete, número ACK, ID de matriz, CRC32
NACK	Tipo de paquete, número ACK, ID de matriz, CRC32
END	Tipo de paquete, número de secuencia, número ACK, ID de matriz, CRC32

12. Integridad de Paquetes

Cada paquete incluirá un valor CRC32 calculado sobre el encabezado y el payload válido del paquete.

Para realizar el cálculo, el campo CRC32 se coloca temporalmente en cero. Luego se calcula el CRC32 sobre:

$$\text{CRC32} = \text{CRC32}(\text{header con CRC32} = 0 + \text{payload válido})$$

Esto permite detectar corrupción no solo en los datos de la matriz, sino también en campos críticos como el tipo de paquete, número de secuencia, número ACK, ID de matriz, número de fragmento, total de fragmentos, tipo de operación y tamaño útil del payload.

El receptor recalculará el CRC32 utilizando el mismo procedimiento y lo comparará con el valor recibido.

12.1. Si el CRC32 coincide

- El paquete se considera íntegro.
- El receptor verifica si el número de secuencia corresponde al paquete esperado.
- Si el paquete es el esperado, se acepta y se actualiza el último número de secuencia recibido correctamente en orden.
- Se envía un ACK acumulativo indicando el último paquete recibido correctamente en orden.

12.2. Si el CRC32 falla

- El paquete se considera corrupto.
- El paquete completo se descarta.
- No se actualiza el número de secuencia esperado.
- El receptor puede reenviar el ACK acumulativo del último paquete recibido correctamente en orden.

13. Números de Secuencia, ACK y Ventana

Cada paquete DATA tendrá un número de secuencia único dentro de la transferencia.

El protocolo utilizará una ventana deslizante Go-Back-N de tamaño configurable. El valor inicial de la ventana será de 8 paquetes.

La ventana permitirá mantener múltiples paquetes en tránsito simultáneamente.

El campo ACK tendrá semántica acumulativa:

- Un ACK con valor n indica que todos los paquetes desde el inicio hasta el paquete n fueron recibidos correctamente y en orden.
- Si el receptor espera el paquete k pero recibe un paquete distinto, lo descarta y vuelve a enviar el ACK del último paquete válido en orden.
- Si se pierde un ACK, el emisor podrá continuar si recibe un ACK posterior con un número mayor.

Los números de secuencia permitirán:

- Detectar paquetes duplicados
- Detectar paquetes perdidos
- Mantener el orden correcto
- Controlar retransmisiones
- Implementar ACK acumulativos

14. Timeouts y Reintentos

El protocolo utilizará un timeout inicial de 500 ms.

Si el emisor no recibe un ACK válido antes de que expire el timeout, retransmitirá todos los paquetes pendientes desde el paquete posterior al último ACK confirmado.

El número máximo de reintentos será 5. Si después de 5 reintentos no se recibe confirmación, la transferencia se marcará como fallida y será cancelada.

15. Estados del Emisor y Receptor

15.1. Estados del Emisor

Estado	Descripción
IDLE	El emisor está esperando una nueva transferencia
START_SENT	El emisor envió START y espera ACK de inicio
SENDING	El emisor envía paquetes DATA dentro de la ventana Go-Back-N
WAITING_ACK	El emisor espera ACKs acumulativos de los paquetes enviados
END_SENT	El emisor envió END y espera ACK final
DONE	La transferencia terminó correctamente
FAILED	La transferencia falló por exceso de reintentos o error irrecuperable

15.2. Estados del Receptor

Estado	Descripción
IDLE	El receptor espera un paquete START
RECEIVING	El receptor acepta únicamente el paquete DATA esperado y descarta paquetes fuera de orden
COMPLETE	El receptor recibió todos los fragmentos correctamente y espera o responde al paquete END
DONE	La transferencia terminó correctamente
FAILED	La transferencia fue cancelada por error de protocolo o fallo de comunicación

16. Secuencia de Comunicación

La transferencia confiable seguirá el siguiente flujo:

1. El servidor principal envía un paquete START.
2. El worker responde con ACK.
3. El servidor principal comienza el envío continuo de paquetes DATA dentro de la ventana Go-Back-N.
4. El worker verifica el CRC32 calculado sobre encabezado y payload.
5. Si el CRC32 es válido, el worker verifica el número de secuencia.
6. Si el paquete recibido es exactamente el paquete esperado:
 - El worker acepta el paquete.
 - Actualiza el último número de secuencia recibido correctamente en orden.
 - Envía un ACK acumulativo con ese número de secuencia.
7. Si el paquete recibido está duplicado o fuera de orden:
 - El worker descarta el paquete.
 - El worker reenvía un ACK acumulativo con el último número de secuencia recibido correctamente en orden.
8. Si un paquete llega corrupto:
 - El worker descarta el paquete completo.
 - El worker no avanza su número de secuencia esperado.
 - El worker puede reenviar el último ACK acumulativo válido.
9. Si ocurre timeout:
 - El servidor retransmite todos los paquetes desde el paquete posterior al último ACK confirmado.
10. Si se supera el máximo de reintentos:
 - El servidor cancela la transferencia.
 - El estado de la transferencia pasa a FAILED.
11. Cuando todos los fragmentos son enviados y confirmados:
 - El servidor envía END.
 - El worker responde con ACK final.

17. Distribución de Matrices

Las matrices 200x200 serán divididas en bloques.

Ejemplo utilizando 4 workers:

- Worker 1 → Filas 0 - 49
- Worker 2 → Filas 50 - 99
- Worker 3 → Filas 100 - 149
- Worker 4 → Filas 150 - 199

Cada worker procesará únicamente las filas asignadas.

18. Serialización de Matrices

Las matrices serán serializadas utilizando arreglos continuos de tipo float32 en formato little-endian.

Python enviará arreglos NumPy hacia C++ mediante pybind11.

C++ convertirá las matrices en secuencias binarias para fragmentarlas en paquetes UDP.

Cada worker reconstruirá los fragmentos recibidos en orden antes de ejecutar las operaciones matriciales.

19. Flujo General del Sistema

1. Python envía matrices a la biblioteca C++
2. El servidor principal divide la matriz
3. La matriz es serializada
4. Los bloques son fragmentados en paquetes UDP
5. Los paquetes son enviados a los workers
6. Los workers verifican integridad
7. Los workers reconstruyen fragmentos recibidos en orden
8. Los workers realizan operaciones matriciales
9. Los resultados son enviados de vuelta
10. El servidor principal reconstruye la matriz final
11. Los resultados son retornados a Python

20. Operaciones Matriciales

Las operaciones iniciales serán:

- Multiplicación de matrices
- Suma matricial
- Promedio matricial

20.1. Distribución de Multiplicación Matricial

La multiplicación matricial distribuida dividirá la matriz A por bloques de filas. La matriz B será replicada en todos los workers para permitir el cálculo paralelo de:

$$C = A \times B$$

Cada worker calculará únicamente las filas asignadas de la matriz resultado.

21. Integración Python y C++

El proyecto utilizará pybind11 para conectar Python con C++.
Ejemplo:

```
import calculator  
  
resultado = calculator.matrix_add(a, b)
```

Python enviará arreglos NumPy hacia C++.
C++ procesará las matrices y devolverá resultados hacia Python.

22. Posibles Mejoras Futuras

- Selective Repeat
- Timeout dinámico
- Multithreading
- Compresión de paquetes
- Balanceo dinámico de carga
- Aceleración GPU
- Paralelismo híbrido CPU/GPU

23. Tecnologías Utilizadas

Las tecnologías utilizadas son las siguientes:

Tecnología	Uso
Python	Red neuronal
PyTorch	Machine Learning
C++	Procesamiento distribuido
UDP	Comunicación
pybind11	Integración Python/C++

24. Conclusión

El proyecto implementa un sistema distribuido de procesamiento matricial utilizando UDP y una capa RDT desarrollada manualmente.

La arquitectura combina Python para definición de redes neuronales y C++ para procesamiento distribuido de alto rendimiento.

La confiabilidad será implementada mediante:

- Go-Back-N
- Ventanas deslizantes
- ACK acumulativos
- CRC32 aplicado sobre encabezado y payload
- Timeouts
- Retransmisiones automáticas
- Máximo de reintentos
- Estados definidos para emisor y receptor

El sistema permitirá distribuir operaciones matriciales entre múltiples workers manteniendo integridad, sincronización y tolerancia básica a fallos.

La arquitectura propuesta servirá como base para futuras extensiones orientadas a computación distribuida y entrenamiento paralelo de redes neuronales.